

Tentu, mari kita bedah kode ini secara mendalam. Kode ini adalah bagian dari logika pemrosesan gambar (*image processing*) yang melakukan operasi matematika dasar (aritmatika) pada dua gambar secara *pixel-by-pixel*.

Berikut adalah penjelasan detail per bagian dan per baris kodenya.

1. Struktur Dasar dan Import Library

C#

```
using System;
using System.Drawing;

namespace MiniPhotoshop.Logic.ImageProcessing
{
    public static class ArithmeticOperations
    {
```

- **using System;**: Mengimpor fungsi dasar C# (seperti tipe data int, Math, dll).
- **using System.Drawing;**: **Sangat penting.** Ini adalah *library* GDI+ di C# yang menyediakan kelas Bitmap (untuk gambar) dan Color (untuk piksel warna).
- **namespace ...**: Mengelompokkan kode ini agar rapi dan tidak bentrok dengan nama kelas lain di proyek.
- **public static class ArithmeticOperations**:
 - **public**: Bisa diakses dari mana saja.
 - **static**: Kita tidak perlu membuat *object* (new ArithmeticOperations()) untuk menggunakan fungsi di dalamnya. Cukup panggil ArithmeticOperations.Add(...).

2. Metode Bantu: Clamp

Metode ini adalah "penjaga keamanan" nilai warna.

C#

```
private static int Clamp(int value)
```

```
{
    if (value < 0) return 0;
    if (value > 255) return 255;
    return value;
}
```

- **Tujuan:** Warna dalam format komputer standar (RGB 8-bit) hanya boleh bernilai antara **0** hingga **255**.
- **Logika:**
 - Jika hasil hitungan matematika negatif (misal: $50 - 100 = -50$), kode ini mengubahnya menjadi **0**.
 - Jika hasil melebihi 255 (misal: $200 + 100 = 300$), kode ini mengubahnya menjadi **255**.
 - Jika aman (0-255), nilai asli dikembalikan.

3. Mesin Utama: PerformOperation

Ini adalah fungsi yang paling kompleks dan cerdas di kode ini. Fungsi ini dibuat agar kita **tidak perlu menulis ulang loop (perulangan) piksel** untuk setiap operasi (tambah, kurang, kali, bagi).

C#

```
private static Bitmap PerformOperation(
    Bitmap imgA, Bitmap imgB, Func<Color, Color, Color> operation)
```

- **Parameter:**
 - imgA, imgB: Dua gambar input.
 - Func<Color, Color, Color> operation: Ini adalah *delegate* (fungsi yang dikirim sebagai parameter). Ini memberitahu mesin: "Saya akan memberimu dua warna, tolong hitung hasilnya sesuai rumus yang nanti saya berikan."

Menentukan Ukuran Kanvas:

C#

```
int newWidth = Math.Max(imgA.Width, imgB.Width);
int newHeight = Math.Max(imgA.Height, imgB.Height);
```

```
Bitmap resultBmp = new Bitmap(newWidth, newHeight);
```

- Kode ini mencari dimensi maksimum. Jika Gambar A ukuran 100x100 dan Gambar B ukuran 200x50, maka hasil akhirnya adalah **200x100**. Ini memastikan tidak ada bagian gambar yang terpotong.
- `new Bitmap(...)`: Membuat kanvas kosong baru untuk menampung hasil.

Looping (Perulangan Piksel):

C#

```
for (int y = 0; y < newHeight; y++)  
{  
    for (int x = 0; x < newWidth; x++)  
    {
```

- Kita menelusuri setiap titik koordinat (x, y) pada kanvas baru dari kiri ke kanan, atas ke bawah.

Pengecekan Koordinat:

C#

```
bool inA = (x < imgA.Width && y < imgA.Height);  
bool inB = (x < imgB.Width && y < imgB.Height);
```

- Karena ukuran gambar mungkin berbeda, kita cek: Apakah koordinat (x, y) saat ini ada di dalam area Gambar A? (`inA`). Apakah ada di dalam Gambar B? (`inB`).

Logika Penggabungan:

C#

```
if (inA && inB)
```

```

    {
        Color cA = imgA.GetPixel(x, y);
        Color cB = imgB.GetPixel(x, y);
        resultBmp.SetPixel(x, y, operation(cA, cB));
    }

```

- **if (inA && inB):** Ini adalah area di mana kedua gambar bertumpuk (overlap).
- Kita ambil warna dari A dan B, lalu jalankan `operation(cA, cB)` (rumus matematika yang dikirim tadi), lalu simpan hasilnya di `resultBmp`.

C#

```

        else if (inA)
        {
            resultBmp.SetPixel(x, y, imgA.GetPixel(x, y));
        }
        else if (inB)
        {
            resultBmp.SetPixel(x, y, imgB.GetPixel(x, y));
        }
        else
        {
            resultBmp.SetPixel(x, y, Color.Black);
        }
    }

```

- **else if (inA):** Jika koordinat ini hanya ada di A (tidak ada di B karena B lebih kecil), salin saja piksel asli A.
- **else if (inB):** Jika hanya ada di B, salin piksel B.
- **else:** Jika (anehnya) tidak ada di keduanya (biasanya tidak terjadi dengan logika `Math.Max` di atas, tapi ini untuk keamanan), isi dengan Hitam.

4. Operasi Penjumlahan (Add)

C#

```

public static Bitmap Add(Bitmap imgA, Bitmap imgB)

```

```

{
    return PerformOperation(imgA, imgB, (cA, cB) =>
    {
        int r = Clamp(cA.R + cB.R);
        int g = Clamp(cA.G + cB.G);
        int b = Clamp(cA.B + cB.B);
        return Color.FromArgb(r, g, b);
    });
}

```

- Memanggil mesin PerformOperation dengan rumus **Penjumlahan**.
- **Efek Visual:** Gambar menjadi lebih terang.
- **Rumus:** $R_{\text{hasil}} = R_A + R_B$.
- Jika Merah A = 200 dan Merah B = 100, hasil = 300. Fungsi Clamp membuatnya jadi 255 (Mentok Putih/Merah terang).

5. Operasi Pengurangan (Subtract)

C#

```

public static Bitmap Subtract(Bitmap imgA, Bitmap imgB)
{
    return PerformOperation(imgA, imgB, (cA, cB) =>
    {
        int r = Clamp(cA.R - cB.R);
        int g = Clamp(cA.G - cB.G);
        int b = Clamp(cA.B - cB.B);
        return Color.FromArgb(r, g, b);
    });
}

```

- Memanggil mesin dengan rumus **Pengurangan**.
- **Efek Visual:** Digunakan untuk mencari perbedaan antar gambar atau menggelapkan gambar secara drastis.
- **Rumus:** $R_{\text{hasil}} = R_A - R_B$.
- Jika hasilnya negatif, Clamp akan membuatnya jadi 0 (Hitam).

6. Operasi Perkalian (Multiply)

Bagian ini agak berbeda karena menggunakan **Normalisasi**.

C#

```
public static Bitmap Multiply(Bitmap imgA, Bitmap imgB)
{
    return PerformOperation(imgA, imgB, (cA, cB) =>
    {
        int r = Clamp((int)((cA.R / 255.0) * (cB.R / 255.0) * 255.0));
        // ... (sama untuk G dan B)
        return Color.FromArgb(r, g, b);
    });
}
```

- **Mengapa dibagi 255.0?**

- Dalam matematika citra, 0 adalah hitam, 1.0 adalah putih.
- Jika kita langsung kalikan integer: $\$200 \times 200 = 40.000\$$ (Sangat jauh di atas 255, jadi putih total). Ini salah.
- Kita ubah ke rentang 0..1 (float). Contoh piksel abu-abu (128) dikali abu-abu (128):
 1. Normalisasi: $\$128/255 \approx 0.5\$$.
 2. Kali: $\$0.5 \times 0.5 = 0.25\$$ (Lebih gelap).
 3. Kembalikan ke 0..255: $\$0.25 \times 255 \approx 64\$$.

- **Efek Visual:** Gambar menjadi lebih gelap, warna putih (1.0) bersifat transparan, warna hitam (0.0) menutupi segalanya.

7. Operasi Pembagian (Divide)

C#

```
public static Bitmap Divide(Bitmap imgA, Bitmap imgB)
{
    return PerformOperation(imgA, imgB, (cA, cB) =>
    {
        double divR = (cB.R == 0) ? 1 : (cB.R / 255.0);
        // ... (variable divG dan divB sama)
    });
}
```

```

    int r = Clamp((int)((cA.R / 255.0) / divR * 255.0));
    // ... (hitung r, g, b)
    return Color.FromArgb(r, g, b);
});
}

```

- **Proteksi Divide By Zero:** Komputer akan *crash* jika kita membagi dengan nol.
 - $(cB.R == 0) ? 1 : \dots$: Jika pembaginya (Warna B) adalah 0 (Hitam pekat), kita anggap nilainya 1 (atau nilai kecil lain) agar tidak error, atau kita biarkan hasilnya jadi sangat besar (putih).
- **Logika:** Kebalikan dari perkalian. Membagi dengan angka desimal (0..1) akan membuat angka menjadi lebih besar (lebih terang).
 - Contoh: $\$0.5 / 0.5 = 1.0\$$ (Putih).
- **Efek Visual:** Gambar menjadi sangat terang, sering digunakan untuk efek *Color Dodge* atau memutihkan background.

Ringkasan Alur Kerja

1. User memanggil `ArithmeticOperations.Add(img1, img2)`.
2. Kode membuat kanvas baru sebesar ukuran maksimal kedua gambar.
3. Kode melakukan *looping* ke setiap piksel.
4. Jika piksel ada di kedua gambar, kode menjalankan logika penjumlahan/pengurangan/dll.
5. Jika piksel hanya ada di satu gambar, kode menyalin piksel tersebut apa adanya.
6. Hasil akhir dikembalikan sebagai Bitmap baru.